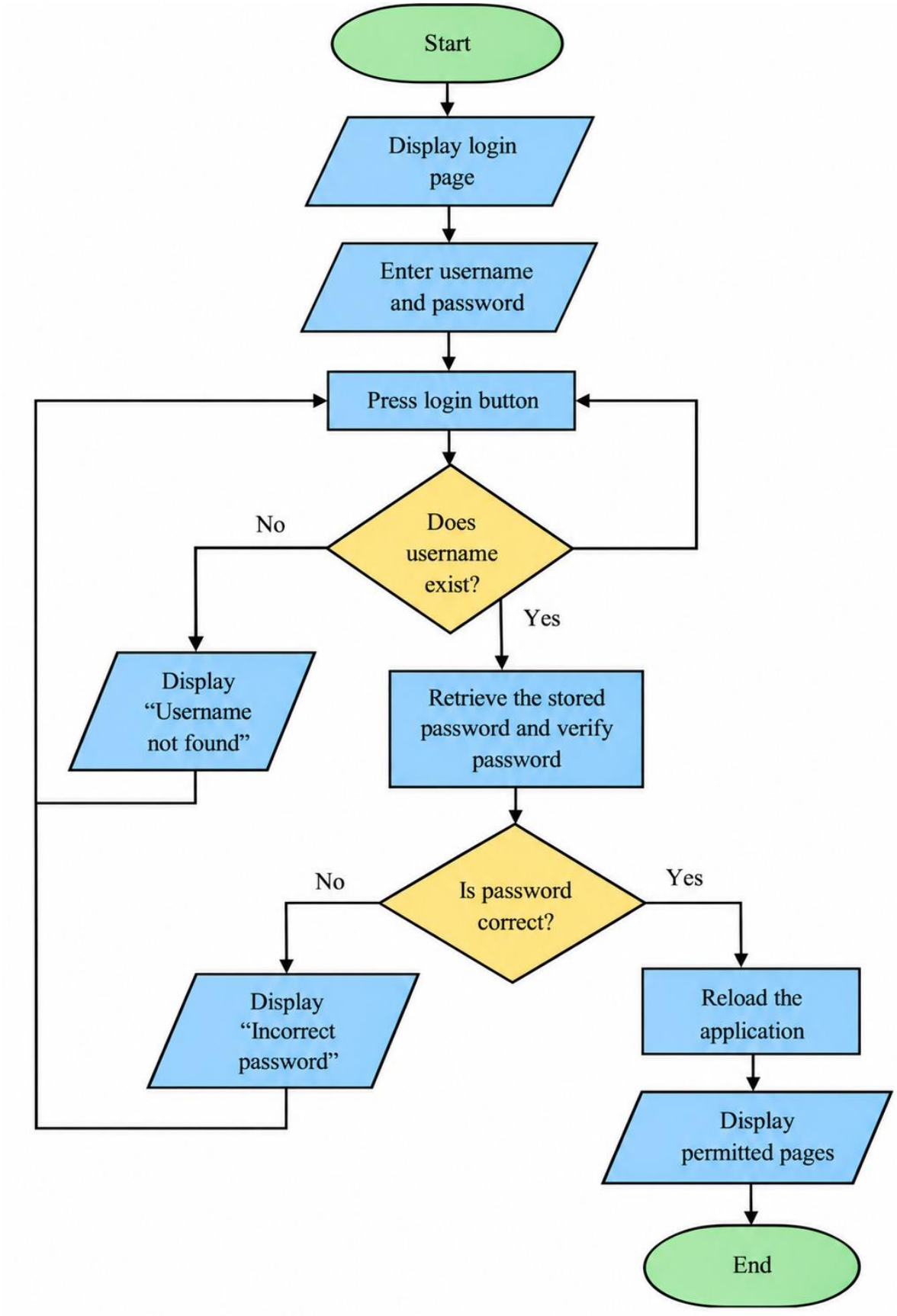
1. Access Control

1. PSUEDOCODE FOR LOGIN

```
BEGIN
  DISPLAY LOGIN PAGE
  INPUT username
  INPUT password
  IF Login button is pressed THEN
    LOAD user accounts from users.csv
    IF username does not exist THEN
      DISPLAY "Username not found"
    ELSE
      RETRIEVE matching user account
      IF account is not active THEN
        DISPLAY "This account is inactive"
     AND
       RETRIEVE stored password
        IF entered_password = stored password  THEN
          SET logged_in = TRUE
          STORE username in session
          STORE user role in session
          RELOAD application
          DISPLAY permitted pages
        ELSE
          DISPLAY "Incorrect password"
        END
```

FLOWCHART:


Start
Display login page
Enter username and password
Press login button
Does username exist?
No
Yes
Display "Username not found"
Retrieve the stored password and verify password
Is password correct?
No
Yes
Display "Incorrect password"
Reload the application
Display permitted pages
End

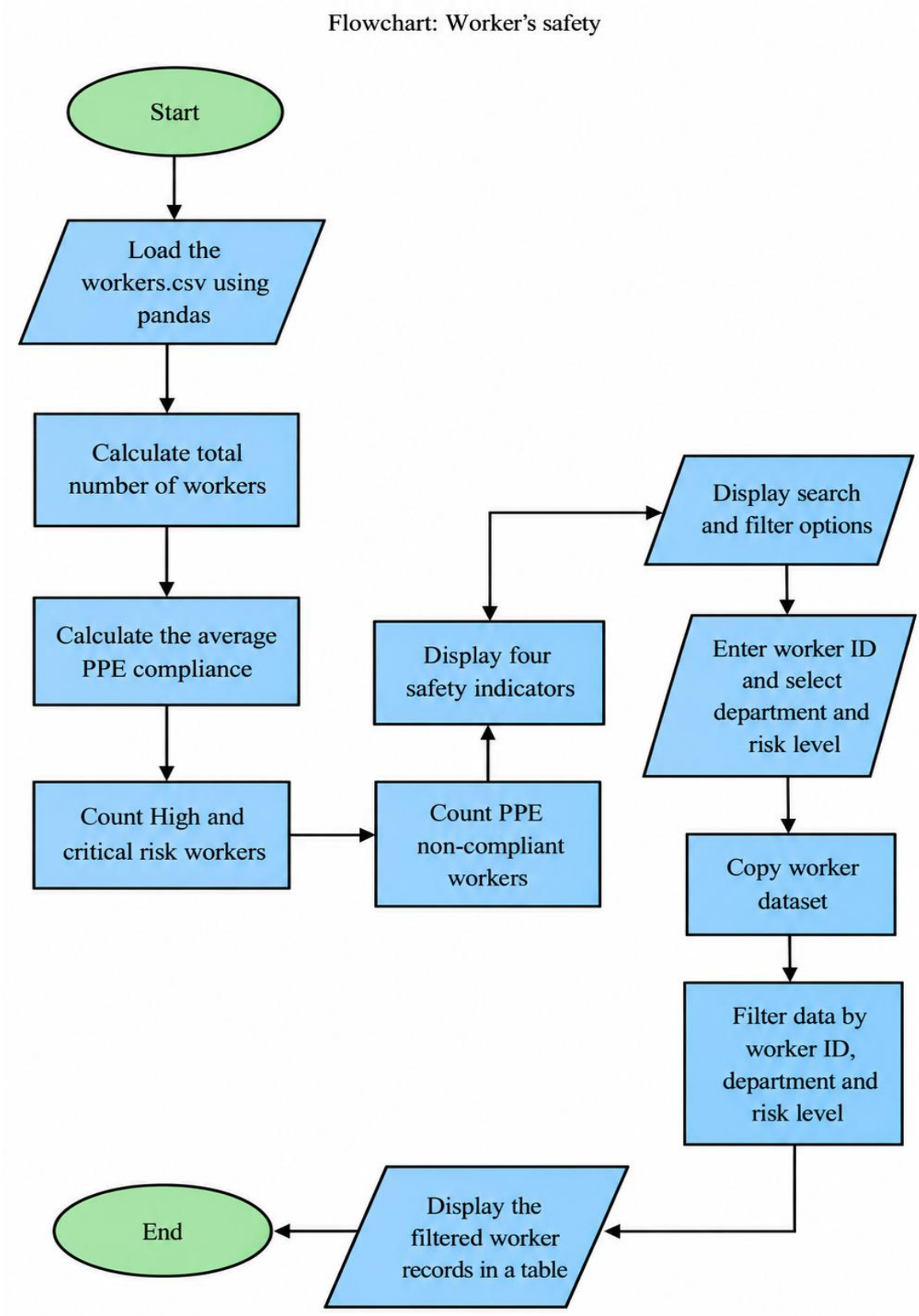
2. Worker Health and Safety Monitoring

Pseudocode

```
BEGIN
  DISPLAY "Worker Health and Safety Monitoring"
  TRY
    LOAD workers.csv using Pandas
    REMOVE extra spaces from column names
    DISPLAY "Safety Overview"
    CALCULATE total_workers = number of worker records
    CONVERT PPE_Compliance_Pct to numeric values
    CALCULATE average_ppe = mean PPE compliance percentage
    CONVERT Risk_Level values to lowercase strings
    COUNT workers with Risk_Level equal to "high" or "critical"
    STORE count as high_risk
    CONVERT PPE_Compliant values to lowercase strings
    COUNT workers whose PPE_Compliant value is "no"
    STORE count as non_compliant
    DISPLAY total_workers
    DISPLAY average_ppe
    DISPLAY high_risk
    DISPLAY non_compliant
    DISPLAY "Search and Filter Workers"
    INPUT search_id
    CREATE department list with "All"
```

```
    ADD available departments
    INPUT selected_department
    CREATE risk_levels list with "All"
    ADD available risk levels
    INPUT selected_risk
    COPY workers into filtered_workers
    IF search_id is not empty THEN
      FILTER filtered_workers by Worker_ID
      MATCH search text without case sensitivity
    ENDIF
    IF selected_department is not "All" THEN
      FILTER filtered_workers by selected_department
    ENDIF
    IF selected_risk is not "All" THEN
      FILTER filtered_workers by selected_risk
    ENDIF
    CALCULATE number of matching worker records
    DISPLAY number of matching workers
    DISPLAY filtered worker records in a table
  CATCH FileNotFoundError
    DISPLAY error message about missing workers.csv
  CATCH KeyError
    DISPLAY error message about missing column
  END TRY
END
```

FLOWCHART


Flowchart: Worker's safety
Start
Load the workers.csv using pandas
Calculate total number of workers
Calculate the average PPE compliance
Count High and critical risk workers
Count PPE non-compliant workers
Display four safety indicators
Display search and filter options
Enter worker ID and select department and risk level
Copy worker dataset
Filter data by worker ID, department and risk level
Display the filtered worker records in a table
End

3. Safety Incidents

flowchart



Pseudocode

Start

Load incident dataset from CSV file

If file doesn't exist, create an empty dataset:

    Ensure all required columns are present

Calculate and display summary metrics (Total, Open, Lost-Time, Critical)

Get user filters (Search text, Department, Severity)

Apply filters and display matching records in a table

Display form inputs for a new safety incident

If "Submit" button is clicked:
    Capture form data as a new record
    Append new record to dataset
    Save updated dataset back to CSV file
    Display success message and refresh app
End

4. Equipment Monitoring

4.1. Equipment Condition

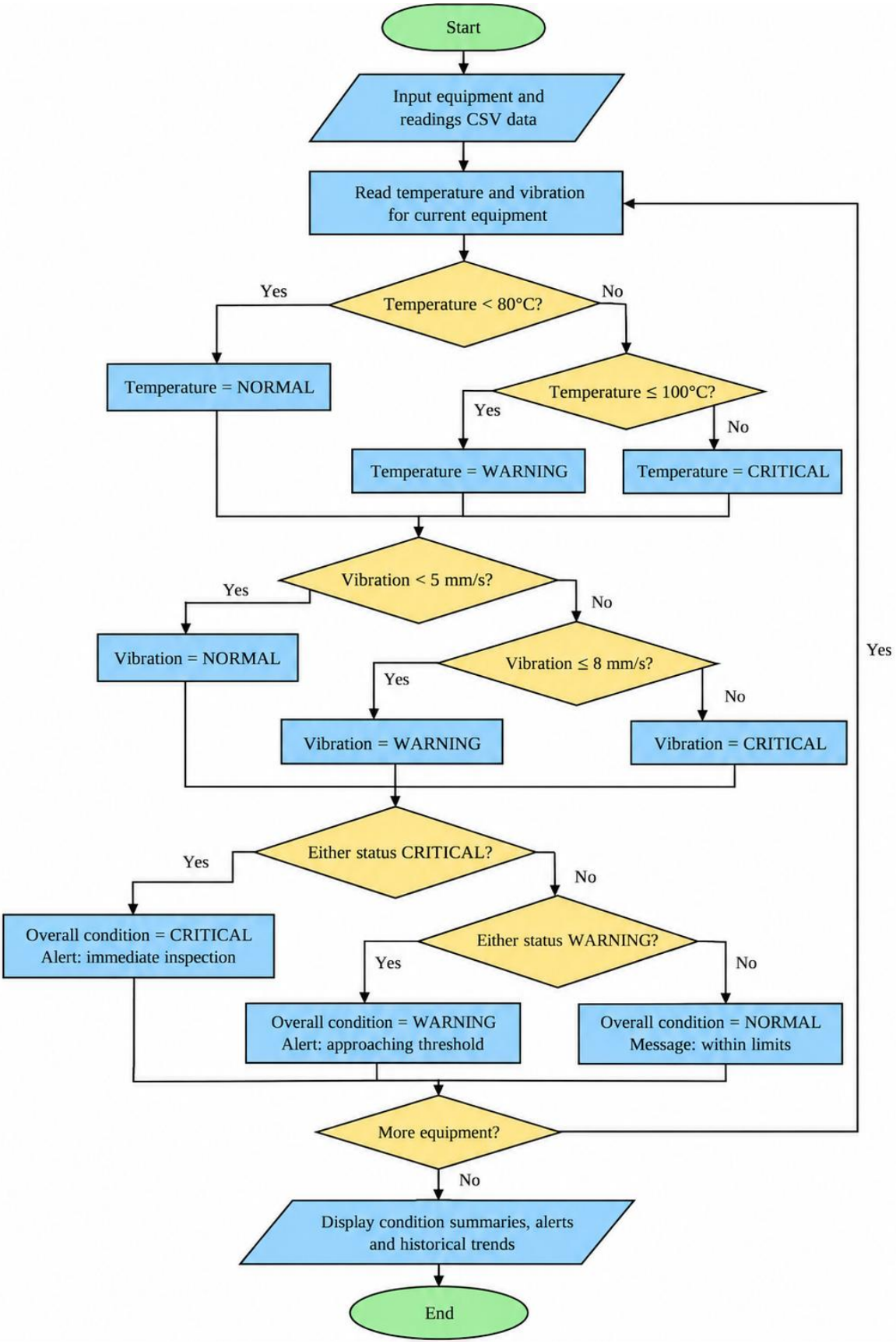
Pseudocode:

```
BEGIN
  LOAD equipment.csv
  LOAD equipment_readings.csv
  FOR each equipment unit
    READ Equipment_ID
    READ Temperature_C
    READ Vibration_mm_s
    IF Temperature_C > 100 THEN
        Temperature_Status = "CRITICAL"
    ELSE IF Temperature_C >= 80 THEN
        Temperature_Status = "WARNING"
    ELSE
      Temperature_Status = "NORMAL"
    END IF

    IF Vibration_mm_s > 8 THEN
      Vibration_Status = "CRITICAL"
```

```
    ELSE IF Vibration_mm_s >= 5 THEN
      Vibration_Status = "WARNING"
    ELSE
      Vibration_Status = "NORMAL"
    END IF
    IF either status is "CRITICAL" THEN
      Overall_Condition = "CRITICAL"
    ELSE IF either status is "WARNING" THEN
      Overall_Condition = "WARNING"
    ELSE
      Overall_Condition = "NORMAL"
    END IF
    GENERATE condition alert if required
  END FOR
  DISPLAY equipment condition summary
  DISPLAY equipment alerts
  SELECT equipment unit
  DISPLAY historical temperature trend
  DISPLAY historical vibration trend
END
```

Flowchart:

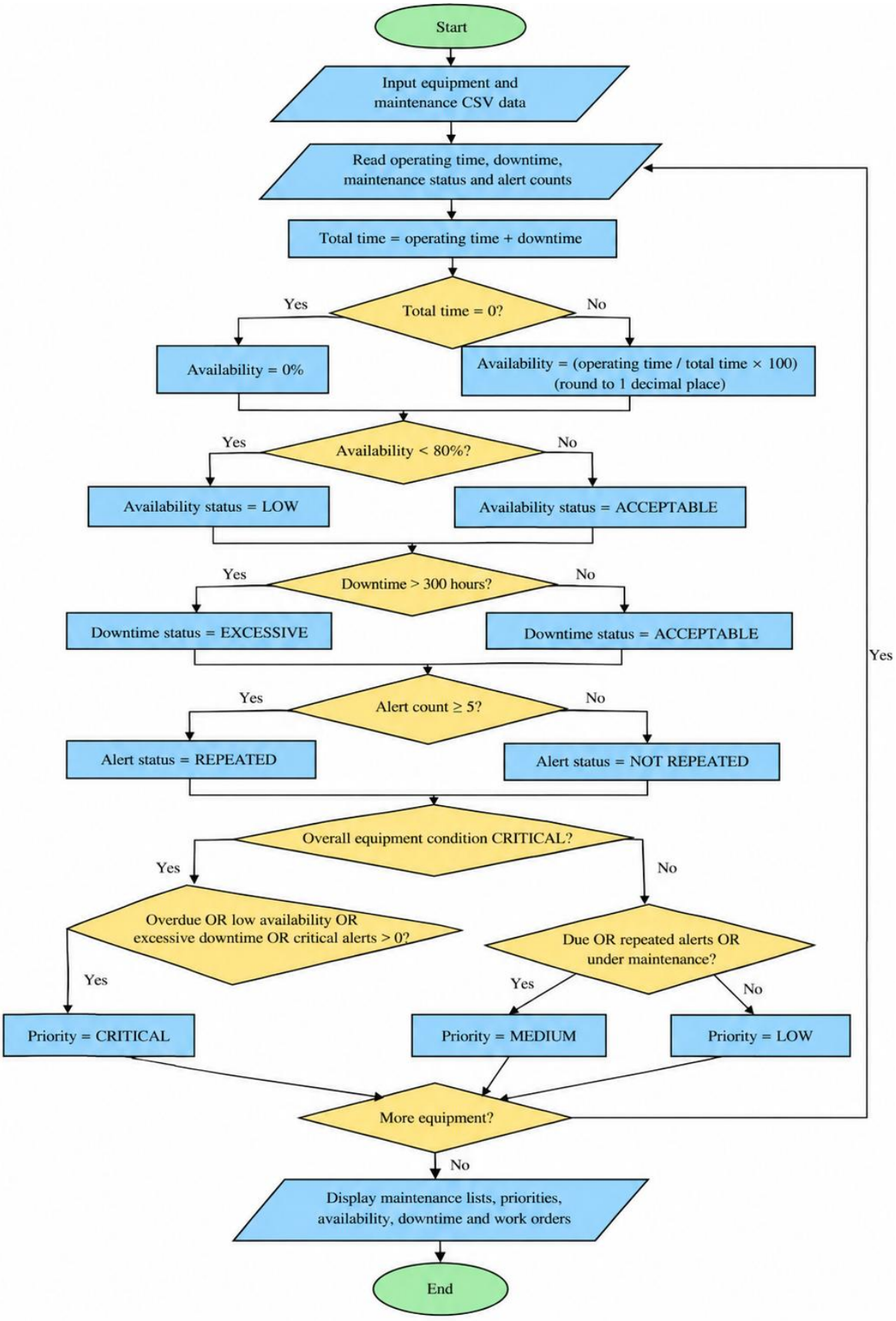

Start
Input equipment and readings CSV data
Read temperature and vibration for current equipment
Temperature < 80°C?
Yes
No
Temperature = NORMAL
Temperature ≤ 100°C?
Yes
No
Temperature = WARNING
Temperature = CRITICAL
Vibration < 5 mm/s?
Yes
No
Vibration = NORMAL
Vibration ≤ 8 mm/s?
Yes
No
Vibration = WARNING
Vibration = CRITICAL
Either status CRITICAL?
Yes
No
Overall condition = CRITICAL
Alert: immediate inspection
Either status WARNING?
Yes
No
Overall condition = WARNING
Alert: approaching threshold
Overall condition = NORMAL
Message: within limits
More equipment?
Yes
No
Display condition summaries, alerts and historical trends
End

4.2. Equipment Maintenance

```
BEGIN
  LOAD equipment.csv
  LOAD maintenance.csv
  FOR each equipment unit
    READ operating time and downtime
    READ maintenance status and alert counts
    READ overall equipment condition
    Total_Time = Operating_Time + Downtime
    IF Total_Time = 0 THEN
      Availability = 0
    ELSE
      Availability = (Operating_Time / Total_Time) * 100
    END IF
    ROUND Availability to 1 decimal place
    IF Availability < 80 THEN
      Availability_Status = "LOW"
    ELSE
      Availability_Status = "ACCEPTABLE"
    END IF
    IF Downtime > 300 hours THEN
      Downtime_Status = "Excessive Downtime"
    ELSE
      Downtime_Status = "Acceptable"
    END IF
    IF Alert_Count_30d >= 5 THEN
      Alert_Status = "Repeated Alerts"
```

```
        ELSE
            Alert_Status = "No Repeated Alerts"
        END IF
        IF Overall_Condition = "CRITICAL" THEN
            Priority = "Critical"
        ELSE IF Maintenance_Status = "Overdue"
            OR Availability_Status = "LOW"
            OR Downtime_Status = "Excessive Downtime"
            OR Critical_Alert_Count_30d > 0 THEN
            Priority = "High"
        ELSE IF Maintenance_Status = "Due"
            OR Alert_Status = "Repeated Alerts"
            OR Maintenance_Status = "Under Maintenance" THEN
            Priority = "Medium"
        ELSE
            Priority = "Low"
        END IF
    END FOR
    IDENTIFY equipment due for maintenance
    IDENTIFY equipment overdue for maintenance
    IDENTIFY equipment under maintenance
    DISPLAY availability and maintenance priorities
    DISPLAY downtime graphs
    DISPLAY maintenance work order history
END
```

Flowchart:


Start
Input equipment and maintenance CSV data
Read operating time, downtime, maintenance status and alert counts
Total time = operating time + downtime
Total time = 0?
Yes
No
Availability = 0%
Availability = (operating time / total time × 100) (round to 1 decimal place)
Availability < 80%?
Availability status = LOW
Availability status = ACCEPTABLE
Downtime > 300 hours?
Downtime status = EXCESSIVE
Downtime status = ACCEPTABLE
Alert count ≥ 5?
Alert status = REPEATED
Alert status = NOT REPEATED
Overall equipment condition CRITICAL?
Overdue OR low availability OR excessive downtime OR critical alerts > 0?
Due OR repeated alerts OR under maintenance?
Priority = CRITICAL
Priority = MEDIUM
Priority = LOW
More equipment?
Display maintenance lists, priorities, availability, downtime and work orders
End

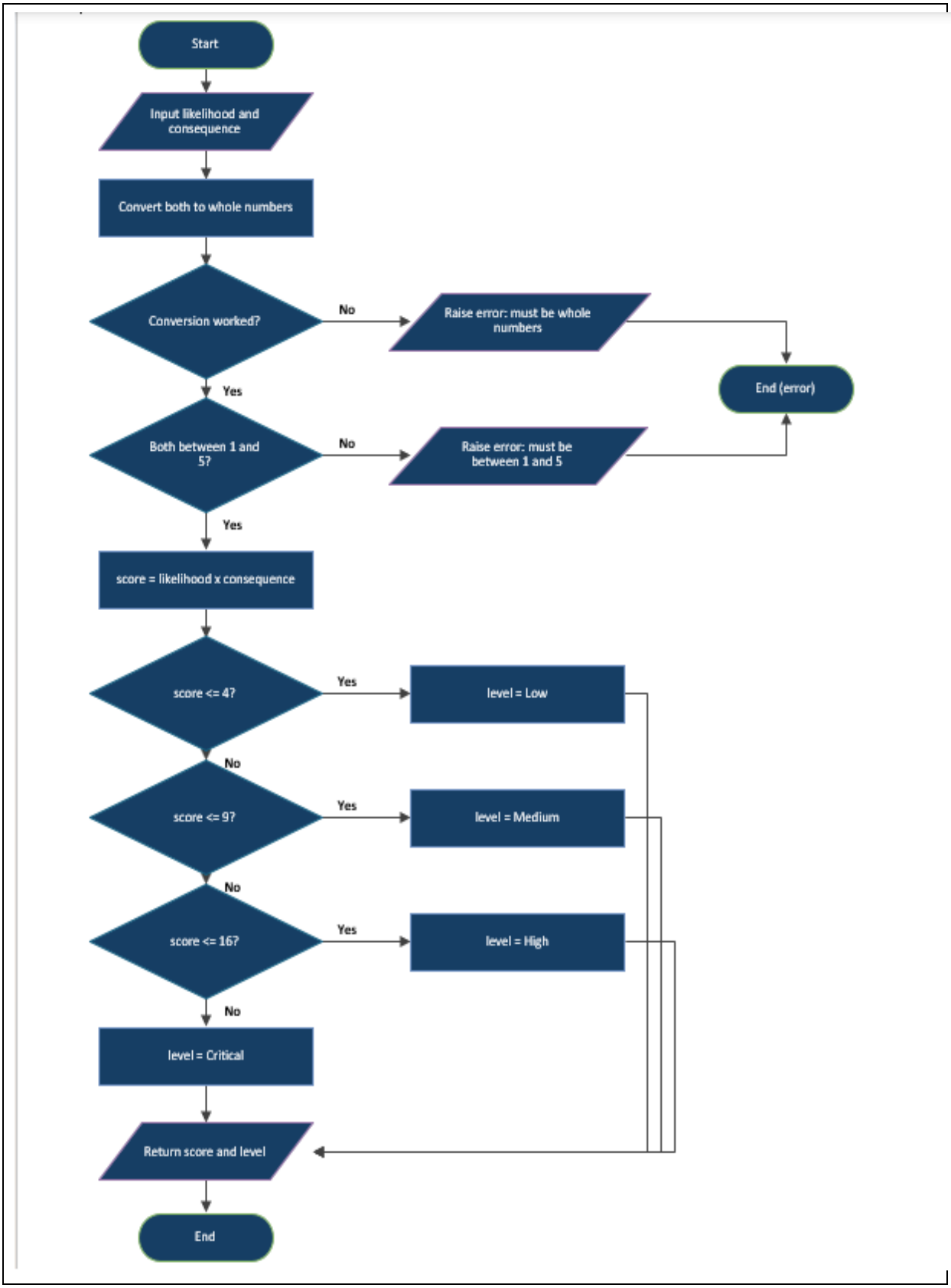
5. Risk Assessment

1. **calculate_risk(likelihood, consequence)**

Pseudocode

```
FUNCTION calculate_risk(L, C)
  TRY convert L and C to integers
  IF conversion fails THEN raise error "must be whole numbers"
  IF L < 1 OR L > 5 OR C < 1 OR C > 5 THEN raise error "must be between 1 and 5"
  score <- L * C
  FOR EACH (top, name) IN BANDS
    IF score <= top THEN RETURN score, name
END FUNCTION
```

Flowchart

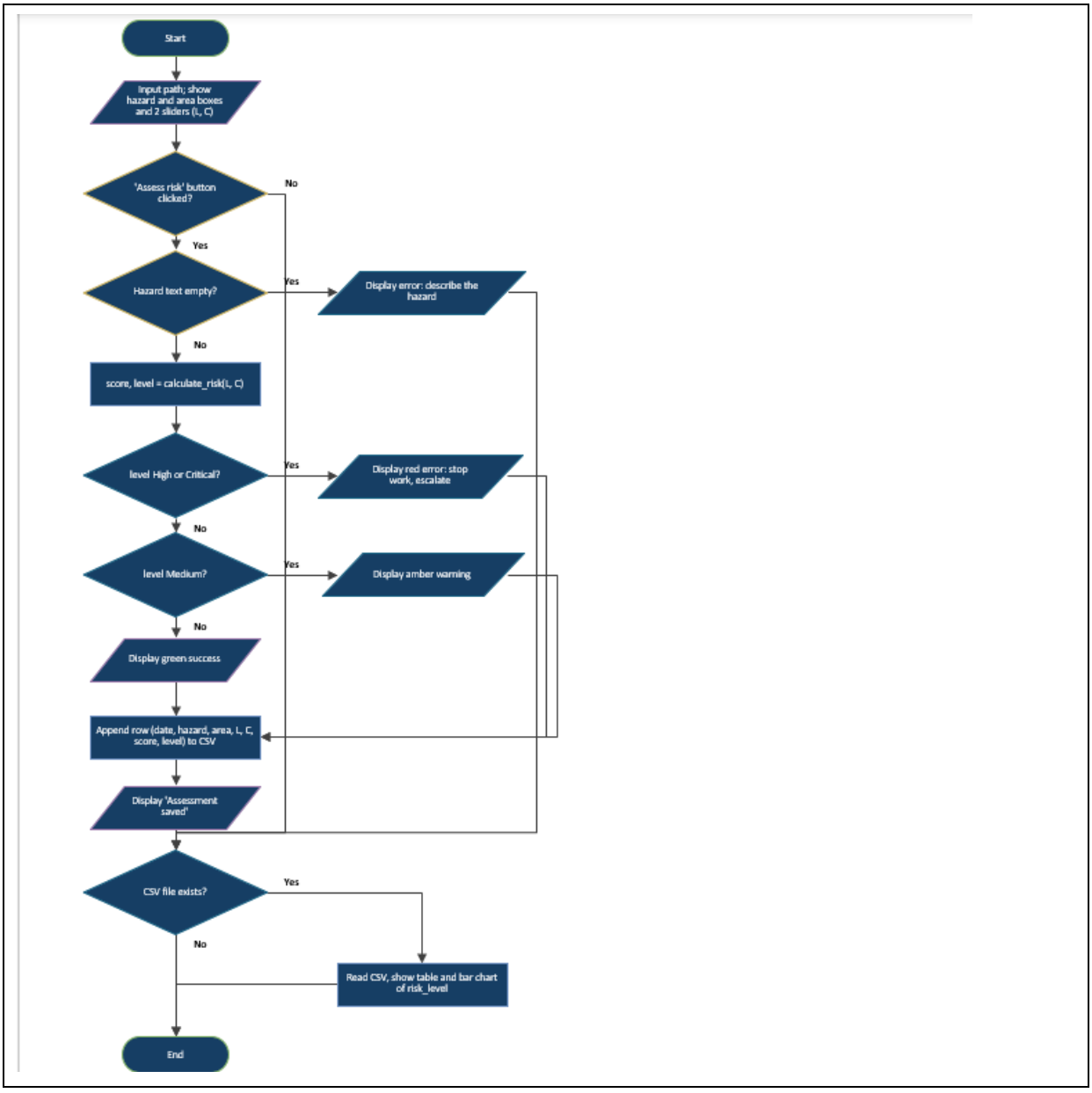


2. **show_risk_page()**

Pseudocode

```
PROCEDURE show_risk_page(path)
  DISPLAY hazard box, area box, L slider, C slider
  IF button "Assess risk" clicked THEN
    IF hazard is empty THEN DISPLAY error
    ELSE
      score, level <- calculate_risk(L, C)
      IF level = "High" OR level = "Critical" THEN DISPLAY red error
      ELSE IF level = "Medium" THEN DISPLAY amber warning
      ELSE DISPLAY green success
      APPEND row(date, hazard, area, L, C, score, level) TO CSV at path
      DISPLAY "Assessment saved"
  IF CSV at path exists THEN
    df <- READ CSV; DISPLAY df
    IF "risk_level" IN df THEN DISPLAY bar chart of its counts
END PROCEDURE
```

Flowchart



3. render(data_folder)

Pseudocode

PROCEDURE render(data_folder)

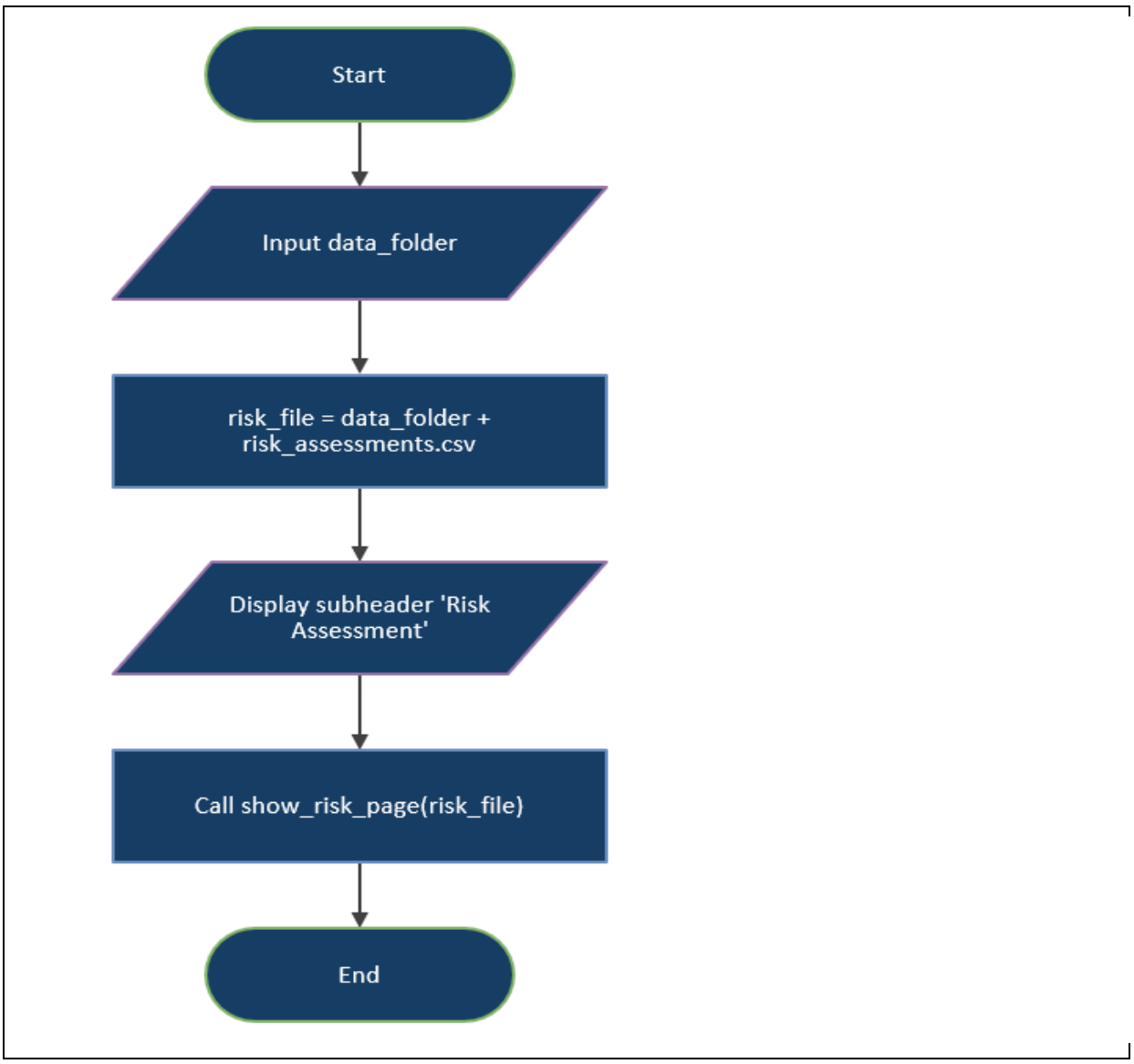
  DISPLAY subheader "Risk Assessment"

  risk_file <- JOIN(data_folder, "risk_assessments.csv")

  CALL show_risk_page(risk_file)

END PROCEDURE

Flowchart

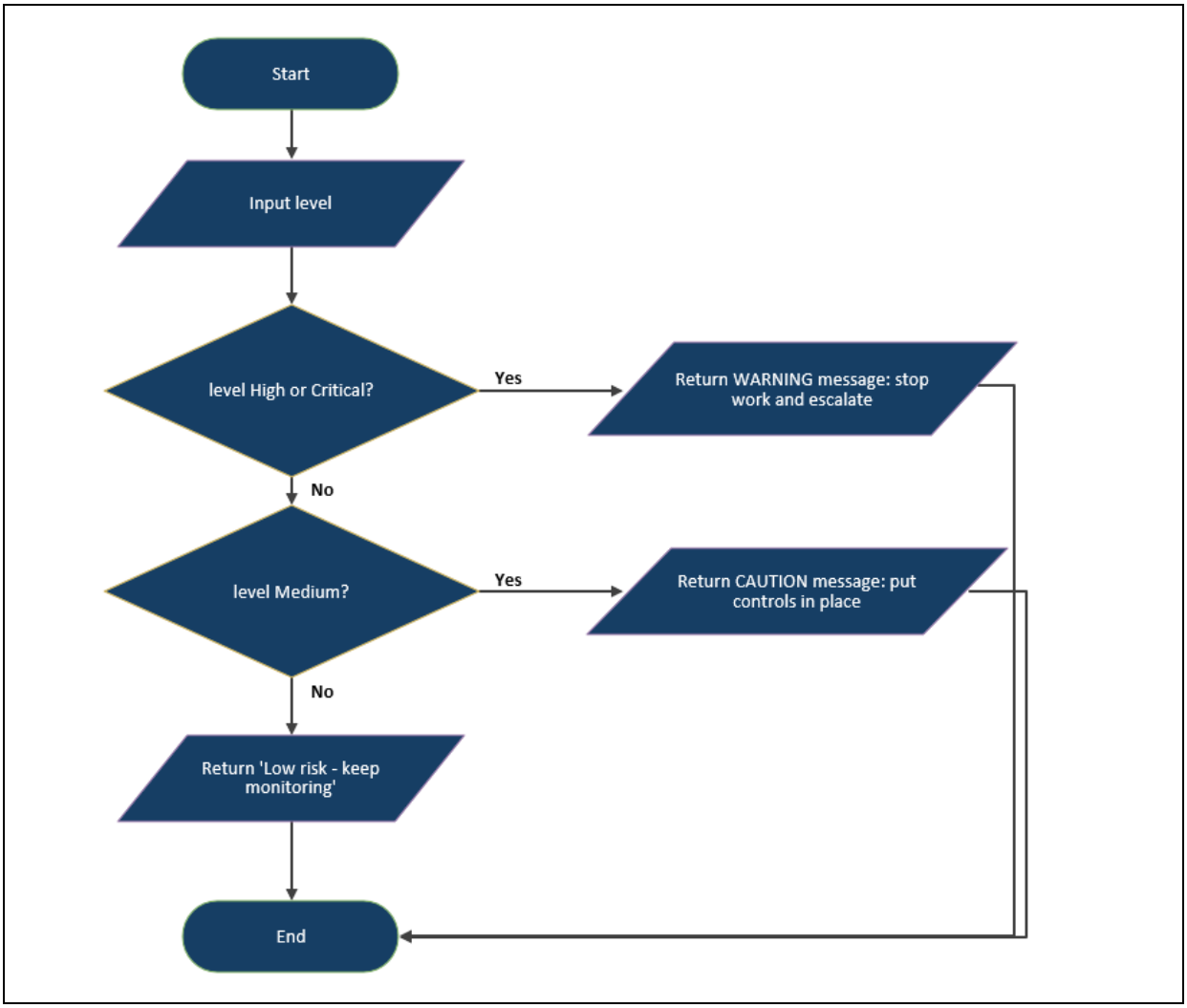


4. risk_warning(level)

Pseudocode

```
FUNCTION risk_warning(level)
  IF level = "High" OR level = "Critical" THEN
    RETURN "WARNING: " + UPPER(level) + " RISK - stop work and escalate to the manager."
  ELSE IF level = "Medium" THEN RETURN "CAUTION: put controls in place."
  ELSE RETURN "Low risk - keep monitoring."
END FUNCTION
```

Flowchart



6. Alerts and Notifications

1. find(df, *keys)

FUNCTION find(df, keys)

FOR EACH k IN keys

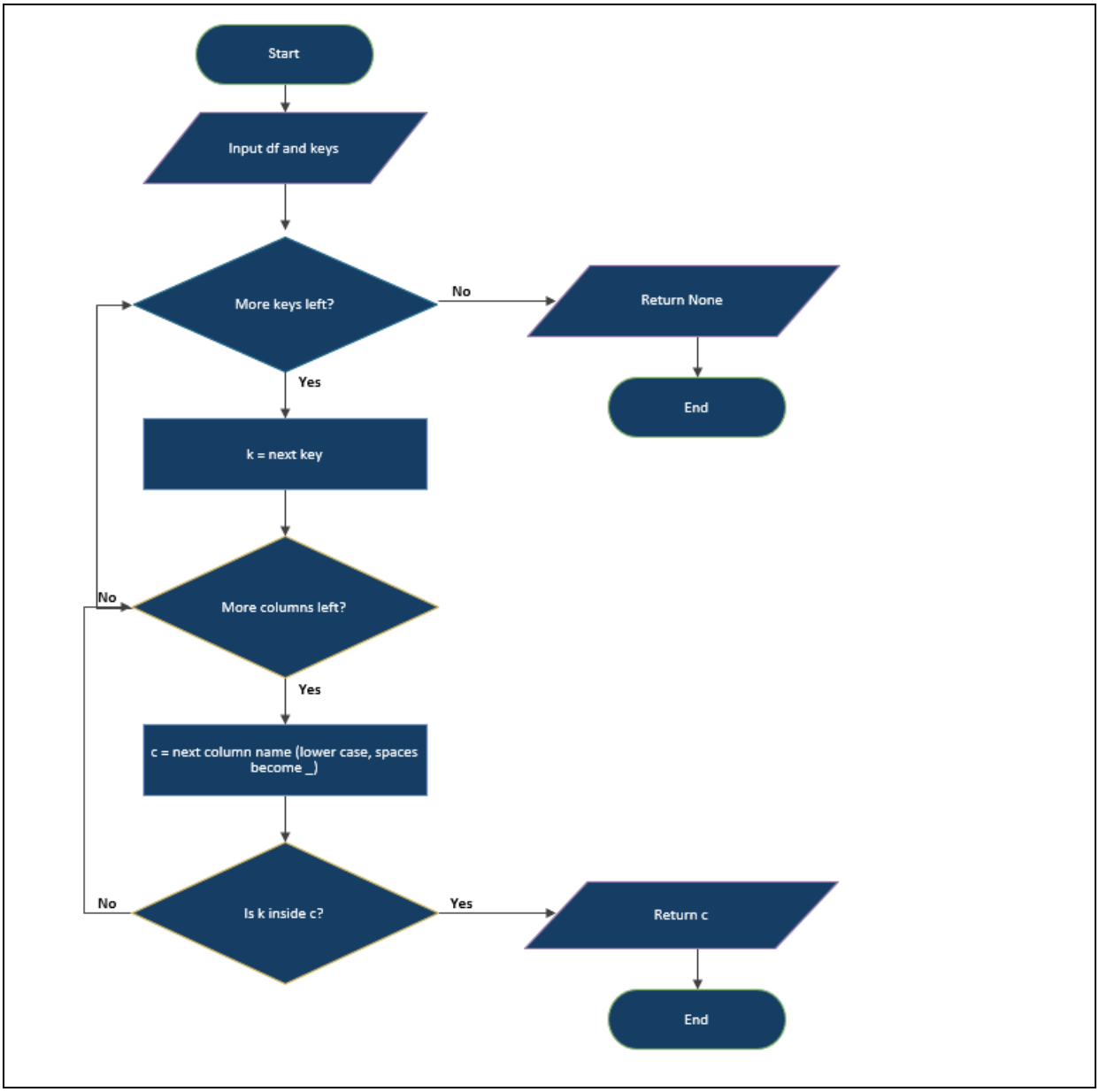
FOR EACH c IN df.columns

IF k IS INSIDE LOWER(c) with spaces replaced by "_" THEN RETURN c

RETURN None

END FUNCTION

Flowchart



2. grade(value, warn, crit, low_is_bad)

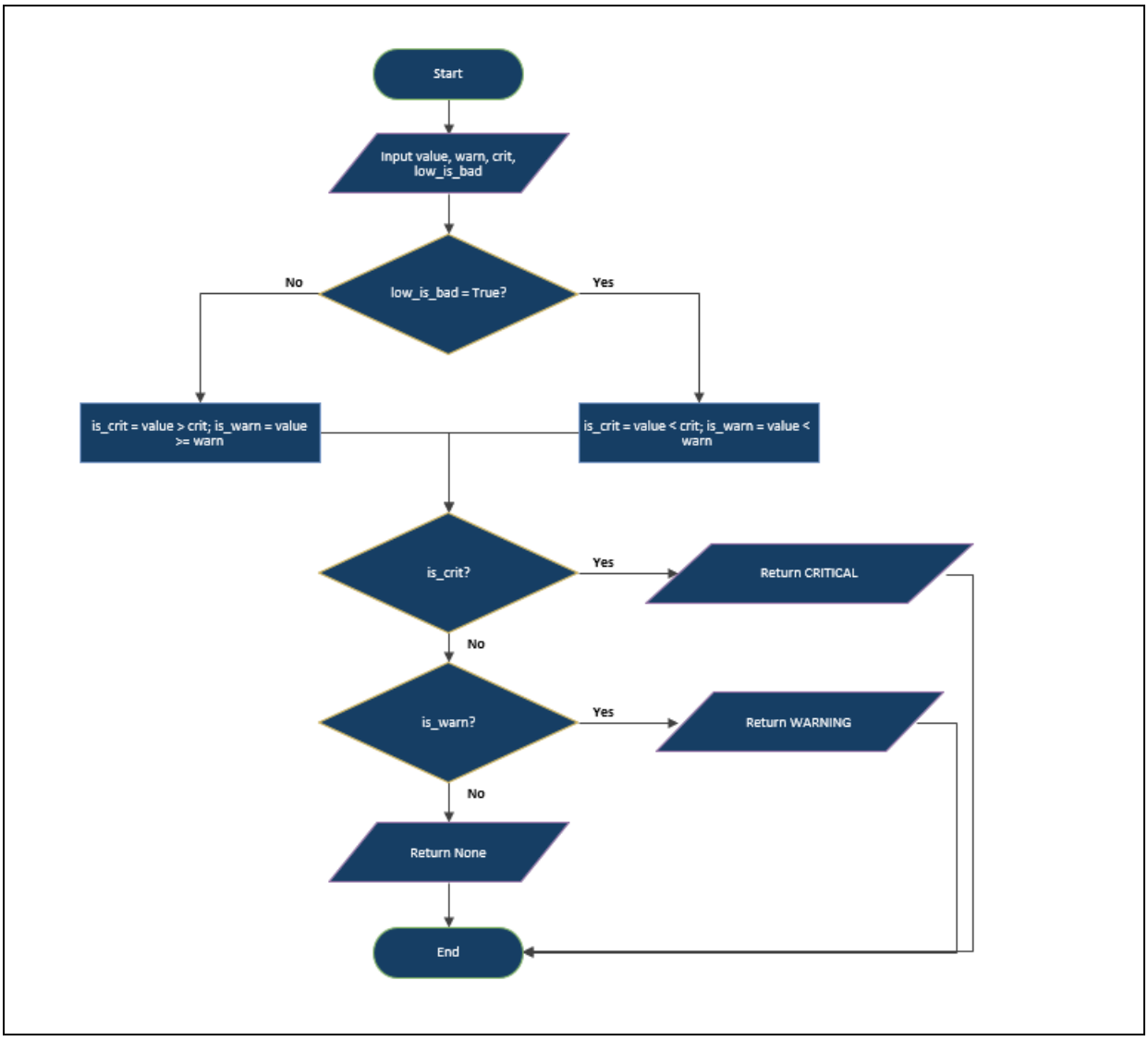
Pseudocode

```
FUNCTION grade(value, warn, crit, low_is_bad)
  IF low_is_bad THEN
    IF value < crit THEN RETURN "CRITICAL"
    ELSE IF value < warn THEN RETURN "WARNING"
  ELSE
    IF value > crit THEN RETURN "CRITICAL"
    ELSE IF value >= warn THEN RETURN "WARNING"
```

RETURN None

END FUNCTION

Flowchart

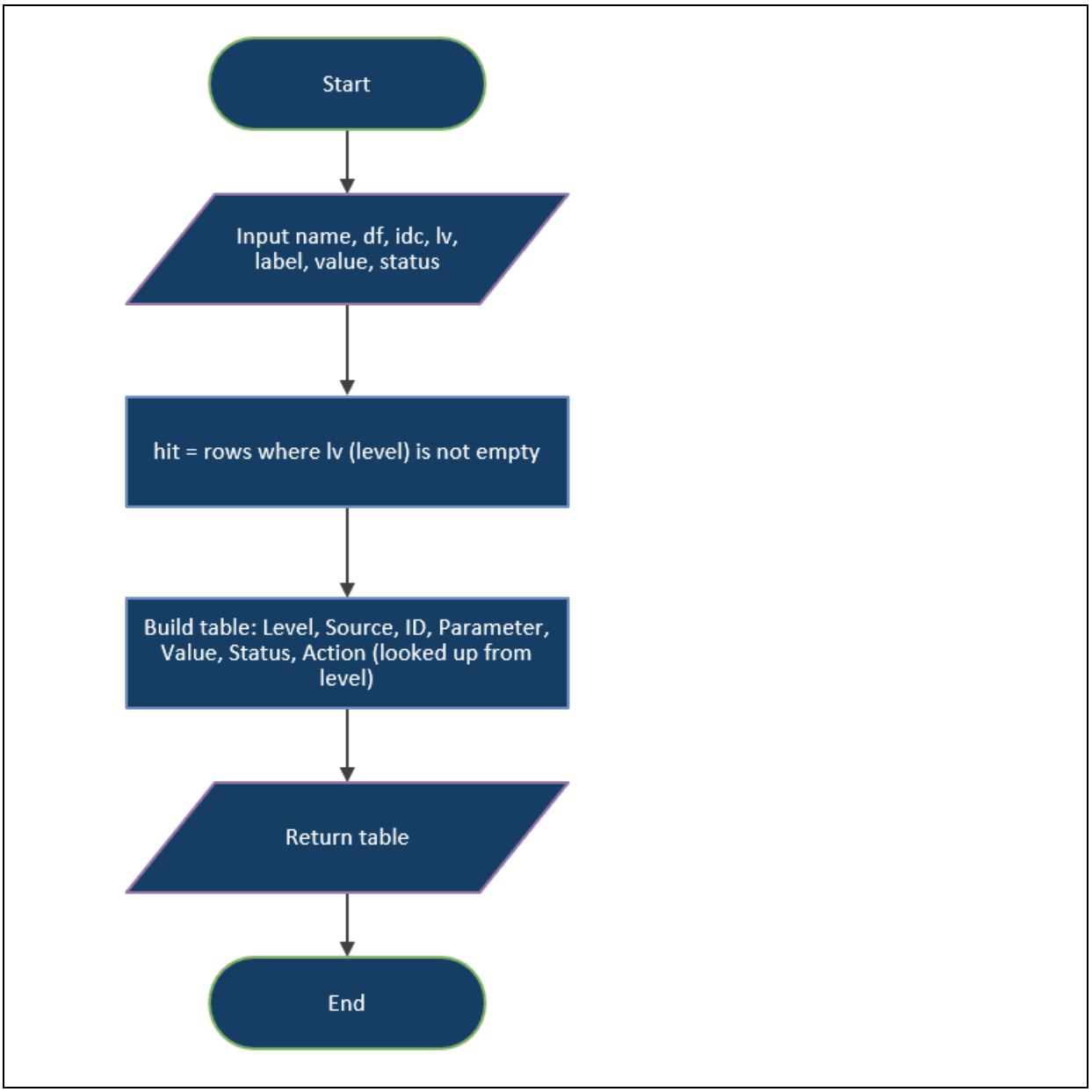


3. mk(name, df, idc, lv, label, value, status)

Pseudocode

FUNCTION mk(name, df, idc, lv, label, value, status)
  hit <- rows WHERE lv is not empty
  RETURN TABLE(Level = lv[hit], Source = name, ID = df[hit][idc],
      Parameter = label, Value = value[hit], Status = status[hit], Action = ACTION[lv[hit]])
END FUNCTION

Flowchart



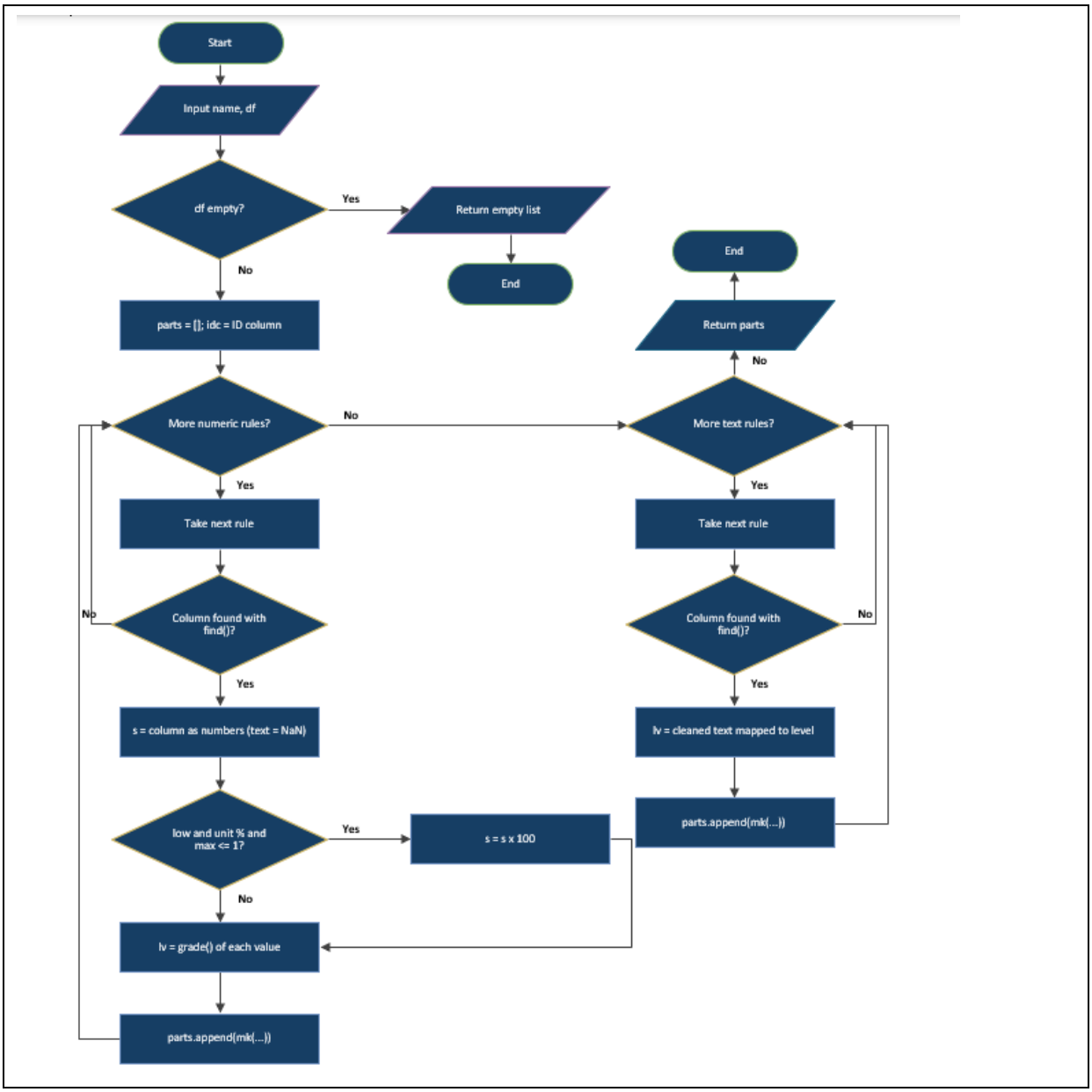
4. scan(name, df)

Pseudocode

```
FUNCTION scan(name, df)
  parts <- empty list
  IF df is empty THEN RETURN parts
  idc <- first existing of equipment_id, worker_id, incident_id, id; else first column
  FOR EACH rule IN NUM_RULES
    col <- find(df, rule.keys); IF col = None THEN CONTINUE
    s <- convert df[col] to numbers
    IF rule.low AND unit = "%" AND MAX(s) <= 1 THEN s <- s * 100
```

```
    lv <- grade of every value in s
    ADD mk(name, df, idc, lv, label, s + unit, status) TO parts
  FOR EACH rule IN TXT_RULES
    col <- find(df, rule.keys); IF col = None THEN CONTINUE
    lv <- MAP(LOWER(STRIP(df[col])), rule.mapping)
    ADD mk(name, df, idc, lv, label, df[col], label + ": " + df[col]) TO parts
  RETURN parts
END FUNCTION
```

Flowchart

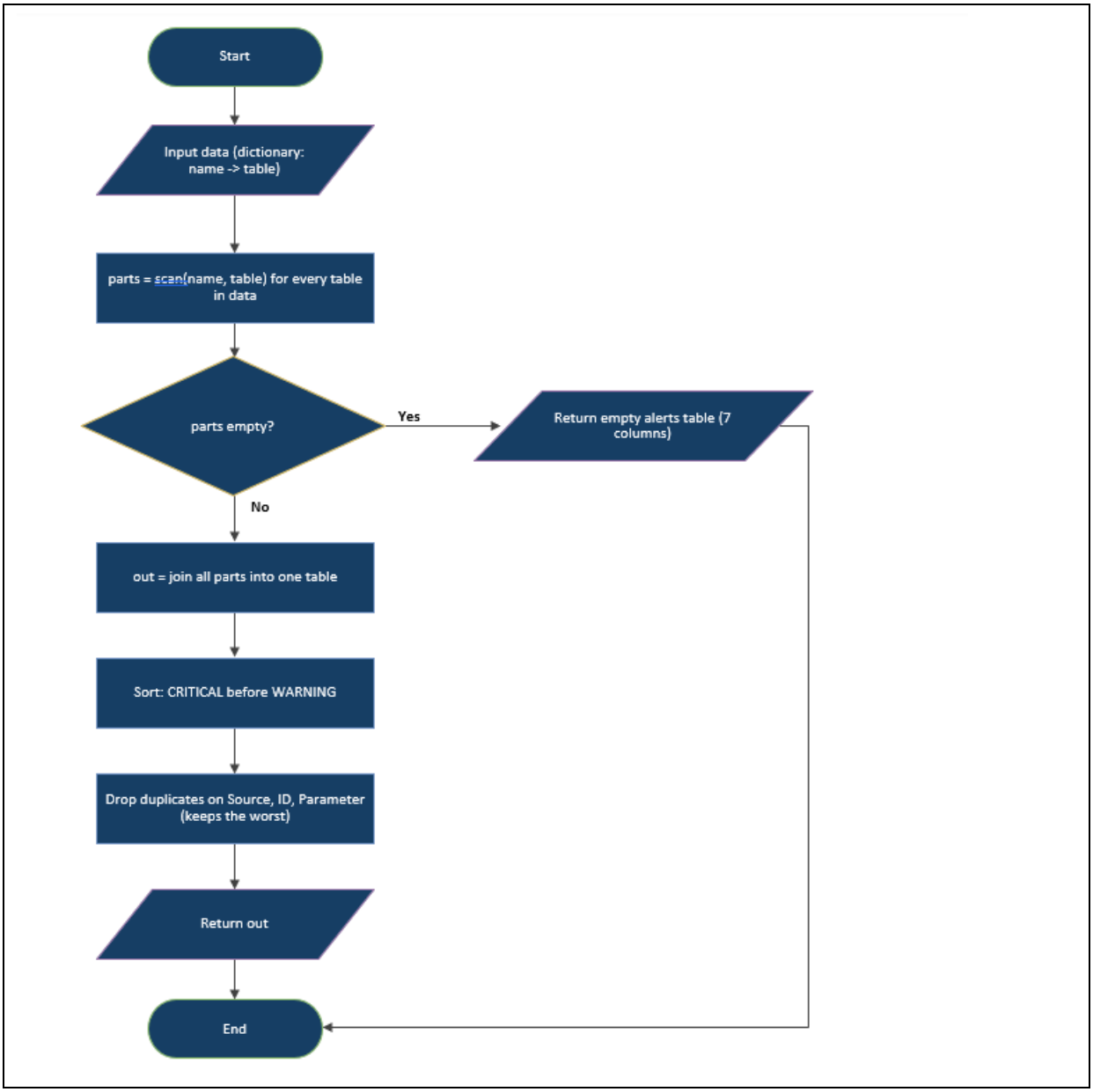


5. generate_alerts(data)

Pseudocode

```
FUNCTION generate_alerts(data)
   parts <- all tables returned by scan(name, table) FOR EACH (name, table) IN data
   IF parts is empty THEN RETURN empty alerts table
   out <- JOIN(parts)
   SORT out BY Level (CRITICAL first)
   REMOVE duplicates ON (Source, ID, Parameter)
   RETURN out
END FUNCTION
```

Flowchart

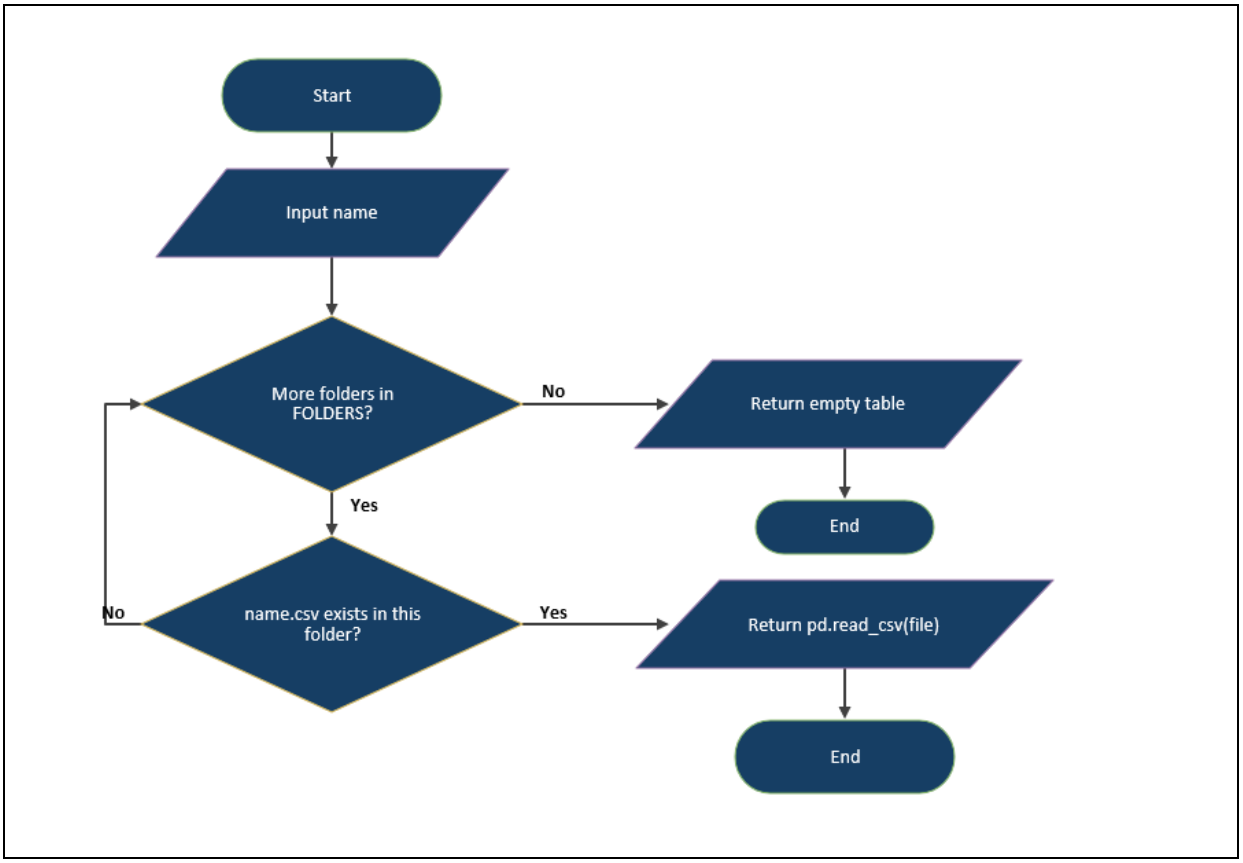


6. load(name)

Pseudocode

```
FUNCTION load(name)
  FOR EACH f IN FOLDERS
    IF f/name.csv exists THEN RETURN READ CSV(f/name.csv)
  RETURN empty table
END FUNCTION
```

Flowchart



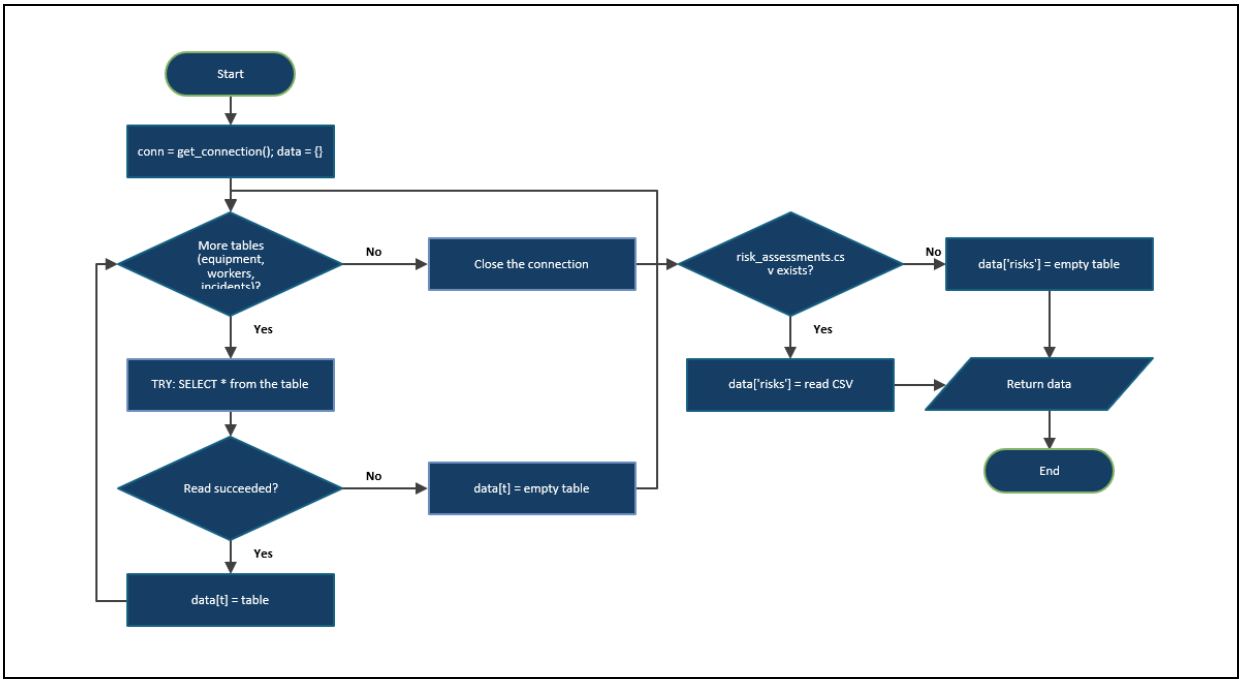
7. load_all_data()

Pseudocode

```
FUNCTION load_all_data()
  conn <- get_connection(); data <- empty dictionary
  FOR EACH t IN ["equipment", "workers", "incidents"]
    TRY data[t] <- READ SQL "SELECT * FROM t"
    ON ERROR data[t] <- empty table
  CLOSE conn
```

IF risk_assessments.csv exists THEN data["risks"] <- READ CSV ELSE data["risks"] <- empty table
RETURN data
END FUNCTION

Flowchart

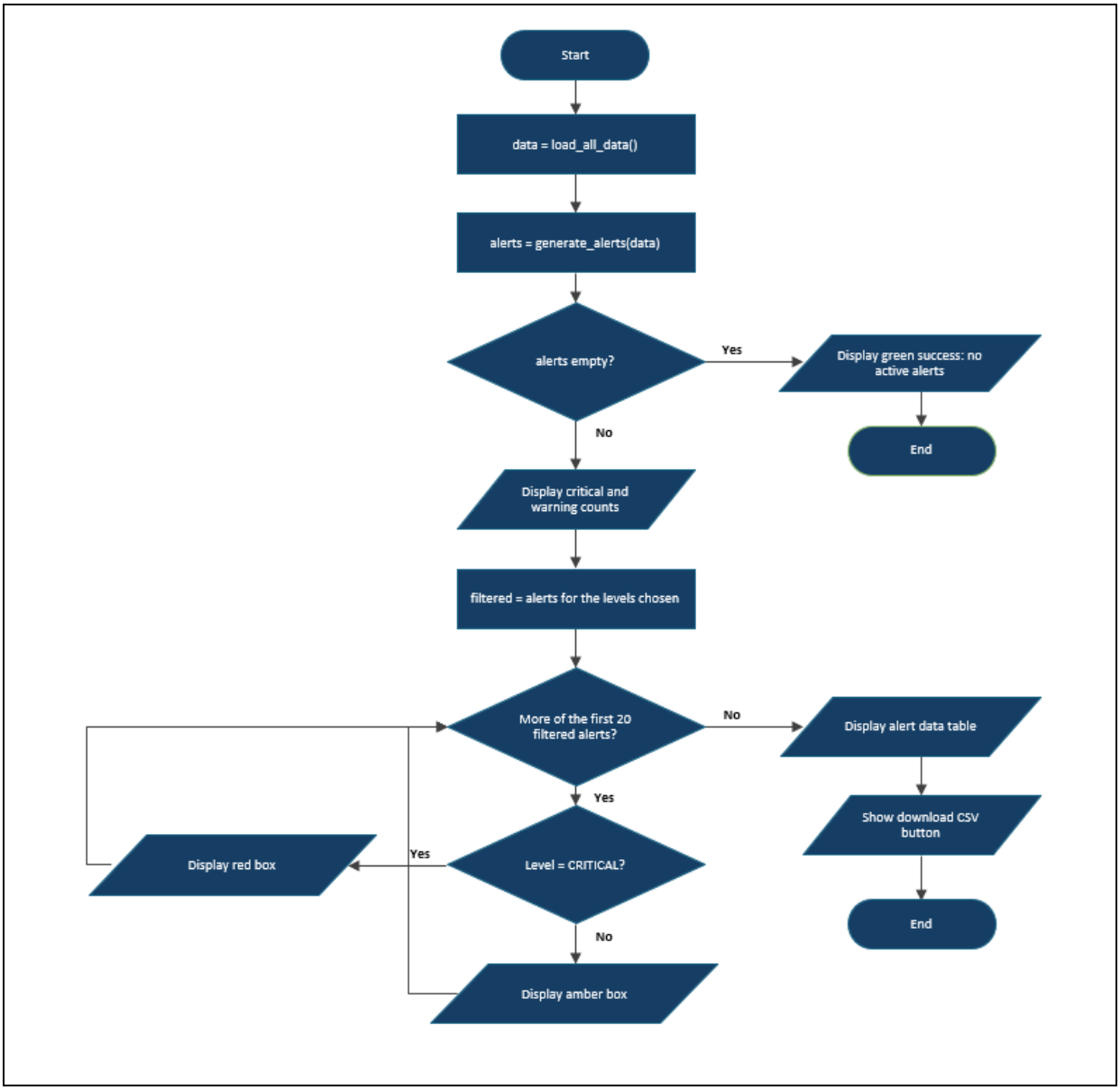


8. Alerts page (main flow)

Pseudocode

data <- load_all_data()
alerts <- generate_alerts(data)
IF alerts is empty THEN DISPLAY "No active alerts"; STOP
DISPLAY count(CRITICAL), count(WARNING)
filtered <- alerts WHERE Level IN chosen levels
FOR EACH a IN first 20 rows OF filtered
IF a.Level = "CRITICAL" THEN DISPLAY red box ELSE DISPLAY amber box
DISPLAY filtered table; OFFER CSV download

Flowchart


Start
data = load_all_data()
alerts = generate_alerts(data)
alerts empty?
Yes
Display green success: no active alerts
End
No
Display critical and warning counts
filtered = alerts for the levels chosen
More of the first 20 filtered alerts?
No
Display alert data table
Show download CSV button
End
Yes
Level = CRITICAL?
Yes
Display red box
No
Display amber box